

Cloud Computing Practicals-

1. Case study on Amazon EC2 and learn about Amazon EC2 web services.

Solution-

Easy Step-by-Step Guide

1. Sign in & IAM Quick-Start
 - Go to <https://console.aws.amazon.com/iam/> and sign in.
 - (Optional) Use your root account or create a single IAM user with **AdministratorAccess**.
2. Create a Simple VPC
 - Go to <https://console.aws.amazon.com/vpc/> → **Start VPC Wizard**.
 - Choose VPC with a Single Public Subnet → Select → Create VPC → OK.
3. Make a Security Group
 - In the VPC console, click **Security Groups** → **Create security group**.
 - Name it BasicSG, pick your new VPC, and click **Create**.
 - Select BasicSG → Edit inbound rules → add two rules:
 1. **SSH** (port 22) from **My IP**
 2. **HTTP** (port 80) from Anywhere (0.0.0.0/0)
 - Save.
4. Launch Your EC2 Instance
 - Go to <https://console.aws.amazon.com/ec2/> → **Launch Instance**.
 - Step 1: Choose Amazon Linux 2 AMI.
 - **Step 2:** Select **t2.micro** (Free Tier).
 - **Step 3:** Under **Network**, pick your new VPC and its subnet.
 - Step 4: Under Security group, choose Select an existing security group → pick BasicSG.
 - Step 5: Click Review and Launch.
5. Create/Select Key Pair & Launch
 - When prompted, Create a new key pair, name it (e.g., MyFirstKey), and download the **.pem** file.
 - Acknowledge and click Launch Instances.
6. Connect & Verify
 - In EC2 console, find your instance's **Public IPv4**.
 - Open a terminal and use these commands-

```
chmod 400 MyFirstKey.pem
ssh -i MyFirstKey.pem ec2-user@ <Your-Instance-Public-IP>
```
 - Install a simple web page and use these commands-

```
sudo yum install -y httpd
sudo systemctl start httpd
echo "Hello from EC2!" | sudo tee /var/www/html/index.html
```

In your browser, go to <http://<Your-Instance-Public-IP>> and see your message.

References-

a) <https://jainakshay781.wordpress.com/wp-content/uploads/2023/02/final-cc-pract-no-1.pdf>

b) <https://anandgharu.wordpress.com/wp-content/uploads/2022/06/final-practical-no-01-2022.pdf>

2. Installation and configure Google App Engine.

Easy Step-by-Step Guide

1. Open GCP Console
 - In your browser, go to [Google Cloud Console](#) and sign in.
2. Create a New Project
 - Click the project selector at top-left → **New Project**
 - Enter a **Project Name** (e.g., my-app-engine) → **Create**
 - Wait a moment, then click the selector again and choose your new project.
3. Enable App Engine
 - In the top search bar, type **App Engine** → select **App Engine**
 - Click Create Application
 - Choose your **Region** → **Next** → on the language screen click **I'll do this later** (we'll use Python).
4. Enable App Engine Admin API
 - Search "App Engine Admin API" in the top bar → click it → **Enable**.
5. Activate Cloud Shell
 - Click the **Activate Cloud Shell** button (top-right; looks like >_)
 - Wait for the shell to start at the bottom of your browser.
6. Clone a Sample "Hello World" App
 - In Cloud Shell, run:
`git clone https://github.com/GoogleCloudPlatform/python-docs-samples`
 - Change directory to the sample app:
`cd python-docs-samples/appengine/standard/hello_world`
7. Run Locally to Test
 - In Cloud Shell, start the dev server:
`dev_appserver.py app.yaml`
 - Note the Local URL (usually `http://localhost:8080`) and open it in the Web Preview (click the preview button in Cloud Shell).
8. Deploy to App Engine
 - Stop the local server with `Ctrl+C`.
 - Deploy your app:
`gcloud app deploy`
 - When prompted, confirm the project number (enter its numeric choice) and press **Y** to continue.
 - Wait for the message Deployed service [default] to [YOUR-APP-URL].
9. View Your Live App
 - In Cloud Shell, run:
`gcloud app browse`
 - Your browser will open your live "Hello World" page.

Summary of commands-

1. Clone sample code

git clone <https://github.com/GoogleCloudPlatform/python-docs-samples>

2. Change into the sample directory

cd python-docs-samples/appengine/standard/hello_world

3. Run locally

dev_appserver.py app.yaml

(Test via Web Preview, then stop with Ctrl+C)

4. Deploy to App Engine

gcloud app deploy

5. Open your deployed app

gcloud app browse

References-

a) <https://jainakshay781.wordpress.com/wp-content/uploads/2023/05/a06-cc-google-app-engine.pdf>

b) <https://anandgharu.wordpress.com/wp-content/uploads/2022/06/final-practical-no-02-2022.pdf>

c) Youtube- <https://www.youtube.com/watch?v=txB0qheXIHE>

3. Creating an Application in Salesforce.com using Apex programming Language.

Step-by-Step Simple Guide

1. Log in and Open the Developer Console
 - Open your browser and log in at <https://login.salesforce.com/>.
 - Once logged in, click on your profile icon (usually your name) in the upper right corner and select **Developer Console**.
2. Create a New Apex Class
 - In the Developer Console, navigate to:
File > New > Apex Class.
 - Name the class as **SimpleHello**.
3. Enter the Minimal Code
 - Copy and paste this small code snippet into the class editor:

```
public class SimpleHello {
    public static void printHello() {
        System.debug('Hello, Salesforce!');
    }
}
```
 - Save the class by clicking **File > Save** (or pressing **Ctrl+S / Cmd+S**).
4. Execute the Code
 - In the Developer Console, navigate to:
Debug > Open Execute Anonymous Window.
 - In the Execute Anonymous Window, type the following line:
SimpleHello.printHello();
 - Click **Execute**.

5. View the Output

- After execution, open the log from the **Logs** panel.
- Look for the debug output which should include:
Hello, Salesforce!
- This confirms that your code ran successfully.

References-

a) <https://jainakshay781.wordpress.com/wp-content/uploads/2023/03/final-cc-practno-31.pdf>

b) <https://anandgharu.wordpress.com/wp-content/uploads/2022/06/final-practical-no-03-2022.pdf>

c) Youtube- <https://www.youtube.com/watch?v=rUeL8kZbjPI>

4. Design and develop custom Application (Mini Project) using Salesforce Cloud.

Easy Step-by-Step Guide

Step 1: Open Salesforce in Lightning Experience

1. Log in to your Salesforce account using Lightning Experience.
 - If you're not already using Lightning, click on your avatar and select **Switch to Lightning Experience**.

Step 2: Launch the Lightning App Creation Process

1. Click on the **App Launcher** (grid icon in the top left corner).
2. Select **View All** to see all available apps.
3. In the quick search field, type **App Manager** and click on it.
4. In the App Manager window, click **New Lightning App**.

Step 3: Configure Your New App

1. Fill in Basic App Details:
 - Enter a **Name** for your app (for example, MyMiniProjectApp).
 - Optionally, fill in any other details like app description, logo, etc.
2. Click **Next**.

Step 4: (Optional) Configure App Options

1. On the **App Options** page, review the settings.
 - These settings are optional; if you do not wish to change them, just click **Next**.

Step 5: (Optional) Add Utilities

1. On the **Add Utility Items** page, you can choose to include extra utility items (such as a Notes or Recent Items component).
 - If you don't need any extra items, simply click **Next**.

Step 6: Add Navigation Items

1. On the **Navigation Items** page, add the items (tabs) that will appear in your app's navigation bar.
 - For instance, you might add **Contacts**, **Accounts**, or any custom objects you plan to use in your mini project.
2. Click **Next**.

Step 7: Assign User Profiles

1. On the **User Profiles** page, select the profiles that will have access to your app.
 - Typically, you would choose profiles (for example, System Administrator or a custom profile) that are part of your organization.
2. Click Save and Finish.

Step 8: Launch and Verify Your App

1. You will now see your new app listed in the **Lightning Experience App Manager** window.
2. Open the **App Launcher** again, and in the search box type a few letters of your app's name to find it along with its logo.
3. Click on your app to open it.
 - You should see the layout and navigation as configured.
4. Verify that you can access and view the app.

References-

a) https://jainakshay781.wordpress.com/wp-content/uploads/2023/04/tejasccprac4_abcdpdf_word_to_pdf-1-5.pdf

b) <https://anandgharu.wordpress.com/wp-content/uploads/2022/06/final-practical-no-04-2022.pdf>

c) Youtube - https://www.youtube.com/watch?v=n4Nx7YD5_mY

(Mini Project)-

5. Setup your own cloud for Software as a Service (SaaS) over the existing LAN in your laboratory. In this assignment you have to write your own code for cloud controller using open-source technologies to implement with HDFS. Implement the basic operations may be like to divide the file in segments/blocks and upload/ download file on/from cloud in encrypted form.

Solution-

1. Verify Hardware and Software Requirements

- Hardware:
 - A Pentium IV (or more advanced) machine with at least two CPU cores
 - Minimum 4 GB RAM (8 GB or more recommended)
 - A local SCSI or RAID disk with unpartitioned space, plus a Gigabit (or faster) Ethernet adapter
- Software:
 - Ubuntu 20.04 installed on a management workstation (if needed)
 - VMware ESXi (version 6.7 or later) installed on the host(s)
 - vCenter (or equivalent management tool) for cluster configuration
 - Open-source libraries and development tools for coding the cloud controller (for example, Java or Python with HDFS support)

2. Install and Configure VMware ESXi on the Host

1. Begin ESXi Installation:
 - Boot your physical machine with the ESXi installer media.
 - Follow the installation prompts:
 - Accept the End User License Agreement (EULA).

- Select the storage device (local SCSI disk or RAID LUN) for installation.
 - Choose your preferred keyboard layout.
 - Set the root password for the ESXi node.
 - Complete the installation and allow the host to reboot.
 - 2. Configure the ESXi Host via the CLI:
 - Once the host reboots, you will see the CLI interface.
 - Configure Management Network:
 - Set the IPv4 address and subnet mask.
 - Set the DNS server address.
 - Restart the management network so that the settings apply.
 - Note that these settings are crucial for remote access and integration with vCenter.
3. Set Up the vCenter and Create a Private Cloud Cluster
1. Access vCenter:
 - Using a web browser on your management workstation, connect to your vCenter URL.
 2. Create a Datacenter:
 - In the vCenter GUI, select “Create Datacenter” and assign it a name.
 3. Create a New Cluster:
 - Inside your newly created Datacenter, choose “Create Cluster.”
 - Assign a name to the cluster.
 - Configure basic settings such as enabling DRS (Distributed Resource Scheduler) and failover options as needed.
 4. Add ESXi Hosts to the Cluster:
 - Click “Add Host” and enter the IP address of your ESXi host.
 - Provide the root credentials you set during installation.
 - Follow the prompts to add the host to your cluster (you may be asked to configure Lock Down mode or add the host to a resource pool).
 5. Review Cluster Configuration:
 - Once added, view the cluster summary to verify that all hosts are online.
 - Check that network and storage resources appear as expected in the cluster view.

4. Develop Your Own Cloud Controller Code with HDFS Integration

In this part, you will write a small program using open-source technologies (such as Java with Hadoop libraries or Python with the appropriate HDFS bindings) that performs basic cloud operations. The operations include splitting files into blocks, encrypting these blocks, and enabling upload/download functionality.

1. Set Up Your Development Environment:
 - On your Ubuntu system, install Java (or Python) and the Hadoop HDFS client libraries.
 - (For Java, ensure that you have the Hadoop core libraries and required dependencies.)
2. Design Your Cloud Controller:
 - File Segmentation:
 - Write code to read an input file and divide it into fixed-size blocks (for example, 64 MB segments).
 - Encryption:
 - Implement a basic encryption mechanism (such as AES) to encrypt each block before uploading.
 - Similarly, code decryption for downloaded blocks.
 - HDFS Operations:
 - Use the HDFS API to upload each encrypted block into HDFS.
 - Implement a method to download these blocks from HDFS and reassemble the file.

3. Write a Sample Pseudocode / Code Outline:

For example, in pseudocode:

// Pseudocode Example for File Segmentation and HDFS Upload

function processFile(filePath):

 blocks = divideFileIntoBlocks(filePath, blockSize=64MB)

 for each block in blocks:

 encryptedBlock = encryptBlock(block)

 uploadToHDFS(encryptedBlock, generateBlockName(filePath, blockNumber))

function retrieveFile(filePath):

 blockNames = listHDFSBlocks(filePath)

 decryptedBlocks = []

 for each blockName in blockNames:

 encryptedBlock = downloadFromHDFS(blockName)

 block = decryptBlock(encryptedBlock)

 append block to decryptedBlocks

 reassembleFile(decryptedBlocks, outputFile=filePath)

- *Note:* Replace pseudocode with actual code in your chosen language. Test encryption and HDFS connectivity separately before integrating.

19. Test Your Cloud Controller Code:

- Run your code to split, encrypt, and upload a test file.
- Then test the download and decryption process to verify that the original file is correctly reassembled.

5. Verification and Final Steps

1. Infrastructure Verification:

- Ensure your ESXi hosts are properly configured and visible in vCenter.
- Check that your private cloud (vSphere cluster) is operational.

2. Cloud Controller Verification:

- Run your cloud controller code on your development machine.
- Verify that file segments are correctly created, encrypted, and transferred to HDFS.
- Test download and decryption to ensure end-to-end functionality.

3. Documentation and Conclusion:

- Document your implementation details, any configurations made on the ESXi and vCenter sides, and sample outputs from your code.
- Conclude by summarizing that you have set up a private cloud infrastructure over the LAN and implemented a basic cloud controller with HDFS integration to handle encrypted file operations.

References-

- a) <https://jainakshay781.wordpress.com/wp-content/uploads/2023/02/final-cc-pract-no-5.pdf>
- b) <https://anandgharu.wordpress.com/wp-content/uploads/2022/05/final-practical-no-05-2022.pdf>